

A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the date.

16/07/2015

# Projet NFE204

Mise en place d'une stack ELK

# Table des matières

|                                 |    |
|---------------------------------|----|
| Introduction.....               | 2  |
| Présentation des logiciels..... | 3  |
| LOGSTASH.....                   | 3  |
| ELASTICSEARCH .....             | 3  |
| KIBANA .....                    | 3  |
| Mise en place.....              | 3  |
| Les données :.....              | 3  |
| SOURCES : .....                 | 3  |
| Format : .....                  | 3  |
| CONTENU : .....                 | 3  |
| Fréquence de collecte : .....   | 5  |
| Volumétrie envisagée.....       | 5  |
| Logstash.....                   | 5  |
| Les entrées .....               | 6  |
| Les filtres .....               | 7  |
| ElasticSearch.....              | 10 |
| Définition des Templates :..... | 10 |
| Kibana .....                    | 12 |
| Mise en pratique .....          | 13 |
| Conclusion .....                | 19 |

# Introduction

Actuellement je suis ingénieur en télécommunication au sein de Bouygues Telecom et j'occupe le poste de responsable validation/support bbox.

Dans le cadre de mon activité, je suis amené à monitorer le déploiement des firmware sur les différents produits fixes proposés par Bouygues Telecom ainsi que le suivi de certains indicateurs de stabilité à partir de paramètres collectés depuis une plateforme de supervision.

Les produits fixes sont les bbox et STB :

- Bbox : box d'accès internet
- STB : décodeur TV

Le monitoring est géré aujourd'hui par la mise en place d'un rapport journalier via excel/vba et diffusé vers les différents chefs de service.

Le but de mon projet est l'automatisation du monitoring des produits et la mise en place d'un suivi en temps réel.

Mon POC (proof of concept) consiste à la mise en place d'un système de monitoring du déploiement de firmware sur les bbox ainsi que le suivi de plusieurs autres paramètres et indicateurs.

Les logiciels choisis pour ce POC sont : Elasticsearch, logstash et Kibana.

# Présentation des logiciels

## LOGSTASH

Logstash est un outil de collecte, analyse et stockage de logs :

- **Collecte** : logstash gère des événements, un événement peut être un message syslog, un csv...
- **Analyse des événements** : Logstash analyse les événements sur lesquels il est en écoute et les met en forme à l'aide de filtres : standardisation de la date, découpage du message... Après filtrage, on obtient un message avec des couples clé-valeur.
- **Stockage** : Logstash exporte ces données sous divers formats : sortie standard, fichier texte, entrée en base de données Elasticsearch.

Dans le cadre de ce projet on traitera plusieurs types de flux (données) en entrée sur lequel on appliquera des filtres à chacun et on les stockera dans Elasticsearch.

## ELASTICSEARCH

ElasticSearch est un moteur de recherche distribué, intégrant une base de données NoSQL.

## KIBANA

Kibana est une interface web permettant de rechercher des infos stockées par Logstash dans ElasticSearch

## Mise en place

### Les données :

#### SOURCES :

Les données sont collectées depuis le serveur d'administration des box tous les jours et déposées sur un serveur partagé.

#### Format :

Les données sont sous format csv.

#### CONTENU :

Dans le cadre de ce POC, 3 types de données seront utilisées :

**Cpereport** : collecte de tous les équipements box et STB (décodeur TV) déployés en production, environ 5 millions d'équipements. Les données sont sous le format ci-dessous :

id;name;ip;created;lastmodified;oui;prodclass;version;serialnumber;dslline;subscriber;population;network;offer;insee;code;ppplogin;firstcontacttime;lastcontacttime;vendorserial;pppservice;imsservice;hsservice;accessservice

Quelques paramètres qui seront utilisés lors de notre analyse via Kibana :

- id : identifiant de la ligne client
- name : identifiant du client
- ip : @ip de la ligne
- created : date de creation de la ligne
- offer : offre client
- ppplogin : compte pppoe du client
- product class : produit déployé chez le client

**CpebootLog** : collecte de tous les équipements qui ont rebooté le jour de la collecte ainsi que le nombre de reboot par équipement. Les données sont sous le format ci-dessous :

Nb\_boot;SN;BOOT\_DATE;FW\_VERSION;POPULATION;PRODUCT\_CLASS;INSEE\_CODE;NAME;FIRST\_CONTACT\_DATE

- Nb\_boot : nombre de reboot du produit le jour de la collecte.
- SN : numéro de série du produit.
- BOOT\_DATE : date de reboot.
- FW\_VERSION : firmware déployé sur l'équipement.
- PRODUCT\_CLASS : type du produit.
- INSEE\_CODE : code postale.

**Bootratio** : collecte du nombre d'équipement (nb\_stb) qui ont rebootés, groupés par type d'équipement (productclass) et nombre de reboot (nb\_boot) par jour ainsi que le nombre totale des équipements (parc) du même type déployés en production.

Exemple de données :

| jour       | prodclass | acces | lowpower | nb_boot | nb_stb | parc   |
|------------|-----------|-------|----------|---------|--------|--------|
| 01/04/2015 | 3504      | xDSL  | N/A      | 1       | 17367  | 274988 |
| 01/04/2015 | 3504      | xDSL  | N/A      | 2       | 2800   | 274988 |
| 01/04/2015 | 3504      | xDSL  | N/A      | 3       | 621    | 274988 |
| 01/04/2015 | 3504      | xDSL  | N/A      | 4       | 223    | 274988 |
| 01/04/2015 | 3504      | xDSL  | N/A      | 5 et +  | 199    | 274988 |

- Jour : Date de la collecte.
- Prodclass : type du produit déployé chez le client
- Accès : type d'accès (adsl/vdsl/ftth...)
- Lowpower : fonction de d'économie d'énergie

- Nb\_boot : nombre de reboot collecté le jour de la collecte
- Nb\_stb : nombre d'équipement qui ont rebooté le jour de la collecte
- Parc : le nombre totale des équipements de type prodclass déployés en production.

Nous verrons dans la suite comment ces informations seront traitées et exploitées.

## Fréquence de collecte :

Les données sont collectées tous les jours à 02h00 et déposées sur un serveur partagé.

## Volumétrie envisagée

Les données collectées dans le cadre du POC font 2 Go mais seront potentiellement plus volumineuses lors de la mise en production de la solution.

## Logstash

Nous allons voir comment installer, configurer et exploiter Logstash

L'installation de Logstash est simple. Le système est écrit en Java, nous aurions besoin un runtime Java récent sur la machine.

Dans le cadre de ce POC, j'ai utilisé la version Logstash 1.4.3 qui m'a semblé être la plus stable, suite au test de la 1.5.2.

- Récupération de la version 1.4.3 depuis <https://download.elastic.co/logstash/logstash/logstash-1.4.3.tar.gz>
- décompresser le zip dans le repertoire /var/opt/Logstash dans lequel on trouvera des sous répertoires tel que :
  - bin: exécutables,
  - lib: librairies.

On va créer dans /opt/Logstash un répertoire 'config' dans lequel on va déposer nos fichiers de configuration qu'on décrira ci-dessous.

Logstash 1.4.3 se lance avec les commandes :

```
./logstash agent
```

Les options disponibles pour l'agent sont :

- -f fichier\_de\_conf : permet de spécifier dans quel fichier l'agent doit lire sa configuration
- -v augmente la verbosité de l'agent
- web : permet de lancer logstash web UI (Kibana) qu'on verra plus loin

Les évènements logstash sont des objets JSON dont lesquels on trouve la structure ci-dessous :

- @timestamp : date de lecture du message
- @type : type du flux
- @source : source du message
- @fields : tableau contenant des champs que l'on veut traiter
- @message : texte du message.

Ces champs peuvent être modifiés, en fonction de notre besoin.

Dans notre répertoire /opt/locstash/config nous allons créer un fichier config.conf dans lequel nous allons définir notre configuration de logstash.

Dans le fichier de configuration on spécifiera :

- les entrées : des fichiers csv dans notre cas,
- les filtres : le traitement qui sera appliqué à nos fichiers
- les sorties : permet de configurer la façon dont logstash écrira les évènements après traitement.

## Les entrées

Les entrées permettent de récupérer les flux qu'on souhaite traiter.

Logstash permet de récupérer plusieurs types d'entrée dans le même fichier de configuration et d'appliquer un traitement spécifique pour chaque type d'entrée. Un type sera défini pour chaque entrée.

Logstash analyse chaque nouvelle ligne du fichier pour chaque entrée, mais n'analysera pas les lignes déjà présentes dans le fichier avant son lancement.

La configuration des inputs dans le cadre de ce projet :

```
input {
  file {
    #cet input concerne les fichiers cpereport.csv
    #on ignore les fichiers de type gz
    'exclude' => ['*.gz']
    #logstash est en écoute sur le répertoire /var/tmp/ELK/cpereport
    'path' => ['/var/tmp/ELK/cpereport/*']
    #On déclare un type à cette entrée
    'type' => 'CPE_REPORT'
  }
  file {
    #cet input concerne les fichiers cpeboot.csv
    #on ignore les fichiers de type gz
    'exclude' => ['*.gz']
    #logstash est en écoute sur le répertoire /var/tmp/ELK/cpeboot
    'path' => ['/var/tmp/ELK/cpeboot/*']
    #On déclare un type à cette entrée
```

```

    'type' => 'CPE_BOOT'
  }
file {
  #cet input concerne les fichiers bootratio.cvs
  #on ignore les fichiers de type gz
  'exclude' => ['*.gz']
  #logstash est en écoute sur le répertoire /var/tmp/ELK/ bootratio
  'path' => ['/var/tmp/ELK/bootratio/*']
  #On déclare un type à cette entrée
  'type' => 'CPE_BOOTRATIO'
}

```

Le type permet d'identifier les entrées afin de lui appliquer un filtre spécifique ainsi qu'une sortie spécifique.

## Les filtres

Les filtres permettent de traiter les événements récupérés en entrée. Chaque filtre est associé à un type d'évènement qu'on a défini en entrée.

La configuration des filtres :

```

filter {
  #suppression des entêtes des fichiers traités
  if [message] =~ /^"id","name"/ {
    drop { }
  }
  if [message] =~ /^"name","offer","network"/ {
    drop { }
  }
  if [message] =~ /^"jour","prodclass"," acces"/ {
    drop { }
  }
  #suppression des lignes vide
  if [message] == "" {
    drop { }
  }
  #application des filtres aux événements de type « CPE_REPORT »
  if [type] == "CPE_REPORT"
  {
    #dans l'élément csv on spécifie les noms de colonne associés aux différents champs, ainsi que le séparateur.
    csv {
      columns =>
      ["id","name","ip","created","lastmodified","oui","prodclass","version","serialnumber","dslline","subs

```



```

criber","population","network","offer","insee","ppplogin","firstcontacttime","lastcontacttime","
vendorserial","pppservice","imsservice","hsservice","accessservice"]
    separator => ";"
}
#dans l'élément mutate, on rajoute un champ received_at qui correspond au timestamp (date de
lecture du message) ainsi qu'un champ received_from qui correspond au nom de la machine.
mutate {
    add_field => [ "received_at", "%{@timestamp}", "received_from", "%{host}" ]
}
# le filtre geoip permet d'ajouter des informations de géolocalisation via une adresse ip. Geoip sera
utilisé dans la géolocalisation des clients.
geoip {
    source => "ip"
    target => "geoip"
}
}
#application des filtres aux évènements de type « CPE_BOOT »
if[type] == "CPE_BOOT"
{
#dans l'élément csv on spécifie les noms de colonne associés aux différents champs, ainsi que le
séparateur.
    csv {
        columns =>
["SN","BOOT_DATE","FW_VERSION","POPULATION","PRODUCT_CLASS","INSEE_CODE","NAME","FIR
ST_CONTACT_DATE"]
        separator => ";"
    }
}
#application des filtres aux évènements de type « CPE_BOOTRATIO »
if[type] == "CPE_BOOTRATIO"
{
#dans l'élément csv on spécifie les noms de colonne associés aux différents champs, ainsi que le
séparateur.
    csv {
        columns => ["bootdate","prodclass","acces","lowpower","nb_boot","nb_stb","parc"]
        separator => ","
    }
}
}

```

Une fois les inputs et les filtres définis, on spécifie via les outputs la façon dont logstash écrira les documents post traitement. On insèrera les documents dans Elasticsearch.

Pour chaque type de données on définira le serveur elasticsearch dans laquelle les documents seront insérés avec ces paramètres : embedded => true vu que le serveur est sur la même machine que logstash, protocol => http, port => 9200 et index/type dans lesquels les documents seront insérés.

La configuration des outputs :

```
output {  
  #pour chaque type de données définit en entrée, on spécifiera un output  
  if [type] == "CPE_REPORT" {  
    #pour le type CPE_REPORT, les documents seront inserés dans elasticsearch  
    elasticsearch { embedded => true  
                  protocol => http  
                  port => 9200  
    #les documents relatives à l'input CPE_REPORT seront indexés dans l'index logstash-cpe-report-  
    #{+YYYY.MM.dd}  
    index => "logstash-cpe-report-#{+YYYY.MM.dd}" }  
    #type : format du document indexé  
    type => CPE_REPORT  
  }  
  if [type] == "CPE_BOOT" {  
    elasticsearch { embedded => true  
                  protocol => http  
                  port => 9200  
    #les documents relatives à l'input CPE_REPORT seront indexés dans l'index logstash-cpe-boot-  
    #{+YYYY.MM.dd}  
    index => "logstash-cpe-boot-#{+YYYY.MM.dd}" }  
    #type : format du document indexé  
    type => CPE_BOOT  
  }  
  if [type] == "CPE_BOOTRATIO" {  
    elasticsearch { embedded => true  
                  protocol => http  
                  port => 9200  
    #les documents relatives à l'input CPE_REPORT seront indexés dans l'index logstash-cpe-bootratio-  
    #{+YYYY.MM.dd}  
    index => "logstash-cpe-bootratio-#{+YYYY.MM.dd}" }  
    #type : format du document indexé  
    type => CPE_BOOTRATIO  
  }  
}
```

# ElasticSearch

## Définition des Templates :

Logstash utilise le timestamp d'un événement pour définir le nom d'index. Par défaut, un index est créé chaque jour. Dans la configuration des outputs logstash on a défini des index, par exemple pour le type CPE\_REPORT, les données seront insérées dans l'index logstash-cpe-boot-%{+YYYY.MM.dd}.

Nous allons maintenant définir un Template pour les documents de chaque type.

Dans les templates on définit le mapping qui sera effectué sur les documents reçu de logstash. Les templates nous permettent de spécifier les types de données, si elles sont indexées, le format et bien d'autres unités de contrôle.

- Template de l'index « logstash-cpe-bootratio-%{+YYYY.MM.dd} » :

```
{
#ce template s'applique à tous les documents dont l'index commence par « logstash-cpe-report »
  "template" : "logstash-cpe-report*",
  "settings" : { "index.refresh_interval" : "5s" },
  "mappings" : {
    "_default_" : {
      "properties" : {
        "@timestamp" : { "type" : "date", "format" : "dateOptionalTime" },
        "@version" : { "type" : "integer", "index" : "not_analyzed" },
        "id" : { "type" : "integer", "index" : "not_analyzed" },
        "name" : { "type" : "string", "index" : "not_analyzed" },
        "ip" : { "type" : "ip", "norms" : { "enabled" : false }, "fields" : { "raw" : { "type" : "string", "index" : "not_analyzed" },
"ignore_above":256}}, "ignore_malformed":true},
        "host" : { "type" : "string", "index" : "not_analyzed" },
        "created" : { "type" : "date", "format" : "YYYY-MM-dd' 'HH:mm:ss" },
        "lastmodified" : { "type" : "date", "format" : "YYYY-MM-dd' 'HH:mm:ss" },
        "oui" : { "type" : "string", "index" : "not_analyzed" },
        "prodclass" : { "type" : "string", "index" : "analyzed", "include_in_all": false },
        "version" : { "type" : "string", "index" : "analyzed" },
        "serialnumber" : { "type" : "long", "index" : "analyzed" },
        "subscriber" : { "type" : "integer", "index" : "analyzed" },
        "population" : { "type" : "string", "index" : "analyzed" },
        "network" : { "type" : "string", "index" : "analyzed" },
        "offer" : { "type" : "string", "index" : "analyzed" },
        "inseecode" : { "type" : "string", "index" : "analyzed" },
        "ppplogin" : { "type" : "string", "index" : "analyzed" },
        "firstcontacttime" : { "type" : "date", "format" : "YYYY-MM-dd' 'HH:mm:ss", "ignore_malformed":true},
        "lastcontacttime" : { "type" : "date", "format" : "YYYY-MM-dd' 'HH:mm:ss", "ignore_malformed":true},
        "vendorserial" : { "type" : "string", "index" : "analyzed", "include_in_all": false },
        "pppservice" : { "type" : "string", "index" : "analyzed", "include_in_all": false },
```

```

    "imsservice" : { "type" : "string", "index" : "analyzed", "include_in_all": false },
    "hsservice" : { "type" : "string", "index" : "analyzed", "include_in_all": false },
    "accessservice" : { "type" : "string", "index" : "analyzed", "include_in_all": false },
    "geoip" : {
      "type" : "object",
      "dynamic": true,
      "path": "full",
      "properties" : {
        "location" : { "type" : "geo_point" }
      }
    },
    "verb" : { "type" : "string", "norms" : { "enabled" : false } }
  }
}
}

```

- Template de l'index « logstash-cpe-ratioboot-%{+YYYY.MM.dd} » :

```

{
  "template" : "logstash-cpe-bootratio*",
  "settings" : { "index.refresh_interval" : "5s" },
  "mappings" : {
    "_default_" : {
      "properties" : {
        "@timestamp" : { "type" : "date", "format" : "dateOptionalTime" },
        "@version" : { "type" : "integer", "index" : "not_analyzed" },
        "bootdate" : { "type" : "date", "norms" : { "enabled" : false }, "format": "dd/MM/YYYY", "index" : "analyzed", "index" : "analyzed" },
        "prodclass" : { "type" : "string", "index" : "not_analyzed" },
        "acces" : { "type" : "string", "index" : "analyzed" },
        "lowpower" : { "type" : "string", "index" : "not_analyzed" },
        "nb_boot" : { "type" : "string", "index" : "not_analyzed" },
        "nb_stb" : { "type" : "integer", "index" : "analyzed" },
        "parc" : { "type" : "integer", "index" : "analyzed" }
      }
    }
  }
}

```

- Template de l'index « logstash-cpe-ratioboot-%{+YYYY.MM.dd} » :

```

{
  "template" : "logstash-cpe-boot*",
  "settings" : { "index.refresh_interval" : "5s" },
  "mappings" : {
    "_default_" : {
      "properties" : {

```

```

"@timestamp" : { "type" : "date", "format" : "dateOptionalTime" },
"@version" : { "type" : "integer", "index" : "not_analyzed" },
"nb_boot" : { "type" : "integer", "index" : "analyzed" },
"SN" : { "type" : "string" },
"BOOT_DATE" : { "type" : "date", "norms" : { "enabled" : false }, "format": "YYYY-MM-dd", "index" : "analyzed", "index"
: "analyzed" },
"FW_VERSION" : { "type" : "string", "index" : "analyzed" },
"POPULATION" : { "type" : "string", "index" : "analyzed", "norms" : { "enabled" : false } },
"PRODUCT_CLASS" : { "type" : "string", "index" : "not_analyzed" },
"INSEE_CODE" : { "type" : "integer", "index" : "analyzed" },
"NAME" : { "type" : "string", "index" : "analyzed", "include_in_all": false },
"FIRST_CONTACT_DATE" : { "type" : "date", "index" : "analyzed", "format": "YYYY-MM-dd" },
"geoip" : { "type" : "object", "dynamic" : true, "path" : "full", "properties" : { "location" : { "type" : "geo_point" } } },
"verb" : { "type" : "string", "norms" : { "enabled" : false } }
}
}
}
}

```

La création des templates s'effectue avec une requête REST:

```

curl -X PUT http://localhost:9200/_template/logstash-cpe-report -d @logstash-cpe-report.json
curl -X PUT http://localhost:9200/_template/logstash-cpe-bootratio -d @logstash-cpe-bootratio.json
curl -X PUT http://localhost:9200/_template/logstash-cpe-boot -d @logstash-cpe-boot.json

```

## Kibana

Kibana est une interface de visualisation des données collectées et stockées dans elasticsearch.

L'installation de kibana est simple :

- Récupération de la version 1.4.3 depuis <https://download.elastic.co/kibana/kibana/kibana-4.1.1-linux-x64.tar>
- décompresser le zip dans le repertoire /var/opt/kibana dans lequel on trouvera des sous répertoires tel que :
  - bin: exécutable,
  - config : fichiers de configuration

Dans le répertoire config, nous allons éditer le fichier kibana.yml afin de configurer l'url d'elasticsearch sur laquelle kibana sera en écoute :

```

# The Elasticsearch instance to use for all your queries.
elasticsearch_url: "http://localhost:9200"

```

On accède à kibana via l'url <http://localhost:5601>

## Mise en pratique

Dans un premier temps nous commençons par lancer logstash, elasticsearch et kibana

```
/opt/logstash-1.4.3/bin$ ./logstash agent -f ../config/elk/logstash.conf web
/opt/elasticsearch/bin$ ./elasticsearch
/opt/kibana/bin$ ./kibana
```

On dépose les fichiers à analyser par logstash dans les répertoires spécifiés dans la configuration des inputs.

Une fois les fichiers déposés, on vérifie dans elasticsearch que les documents ont été bien insérés avec le format spécifié.

La figure ci-dessous montre la création de l'index « logstash-cpe-bootratio-2015.07.13 » et l'insertion des documents relative aux lignes du fichier csv initial et conformément au template « logstash-cpe-bootratio » qu'on a créé :

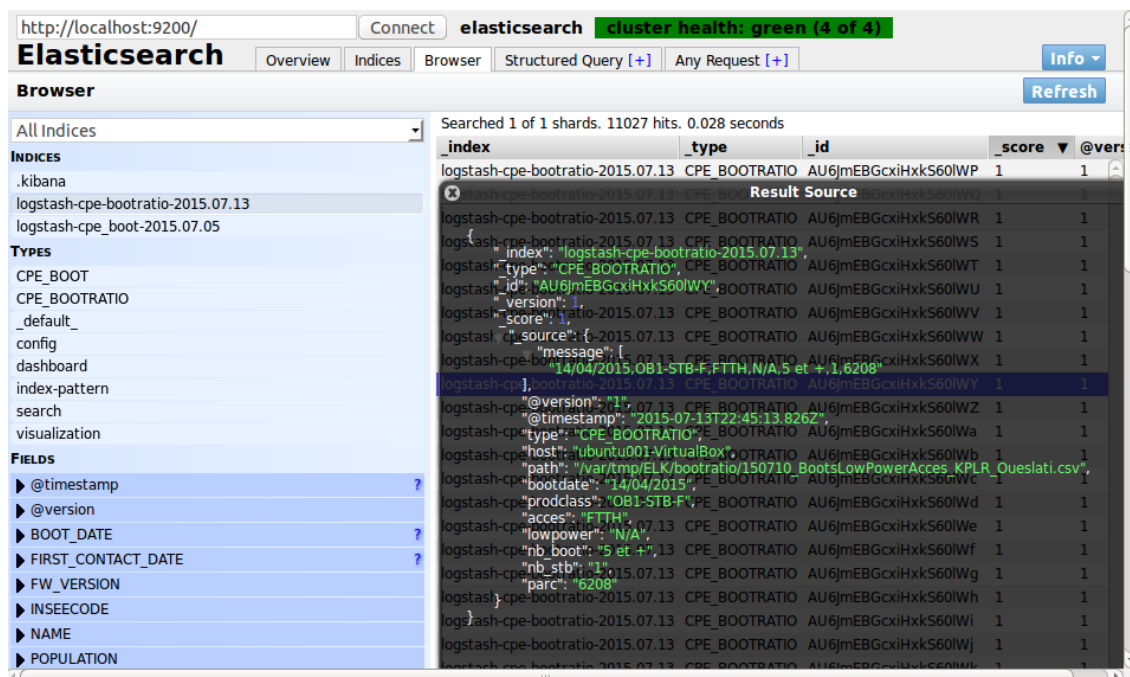


Figure 1 : Insertion des document dans Elasticsearch

Il suffit maintenant de se rendre sur l'interface de kibana <http://localhost:5601> pour commencer à créer nos graphiques et indicateurs.

Dans un premier temps, nous allons configurer dans quel index Elasticsearch nous allons récupérer nos données. Nous avons stocké nos données de bootratio dans un index dont le pattern est : "logstash-cpe-bootratio-%{+YYYY.MM.dd}" », ce qui veut dire que Logstash créera un nouvel index chaque jour. Nous allons reporter ce pattern dans Kibana en spécifiant comme « Time-field name » l'index bootdate qui correspond à la date des reboot de nos équipements (figure ci-dessous). Une fois l'index enregistré, nous pouvons commencer à créer des panels de restitution.

Discover Visualize Dashboard Settings

Indices Advanced Objects About

Index Patterns

**Warning** No default index pattern. You must select or create one to continue.

## Configure an index pattern

In order to use Kibana you must configure at least one index pattern. Index patterns are used to identify the Elasticsearch index to run search and analytics against. They are also used to configure fields.

☒ Index contains time-based events  
☐ Use event times to create index names

**Index name or pattern**  
 Patterns allow you to define dynamic index names using \* as a wildcard. Example: logstash-\*

logstash-cpe-bootratio\*

**Time-field name** [refresh fields](#)  
 bootdate

Create

Figure 2 : Création d'index pattern on peut visualiser la liste des index

Index Patterns

+ Add New

★ logstash-cpe-bootratio\*

★ logstash-cpe-bootratio\*

This page lists every field in the logstash-cpe-bootratio\* index and the field's associated core type as recorded by Elasticsearch. While this list allows you to view the core type of each field, changing field types must be done using Elasticsearch's Mapping API.

Fields (22) Scripted fields (0)

| name       | type    | format | analyzed | indexed | controls |
|------------|---------|--------|----------|---------|----------|
| _source    | _source |        |          |         |          |
| _index     | string  |        |          |         |          |
| bootdate   | date    |        | ✓        | ✓       |          |
| prodclass  | string  |        |          | ✓       |          |
| parc       | number  |        | ✓        | ✓       |          |
| @version   | string  |        |          | ✓       |          |
| acces      | string  |        | ✓        | ✓       |          |
| lowpower   | string  |        |          | ✓       |          |
| _type      | string  |        |          | ✓       |          |
| @timestamp | date    |        |          | ✓       |          |
| _id        | string  |        |          |         |          |
| nb_stb     | number  |        | ✓        | ✓       |          |

Figure 3 : les fields de notre index

Dans le cadre de notre analyse nous aurions besoin de rajouter un nouvel index calculé à partir de deux index présents dans le document, il s'agit de calculer le ratio des box qui ont rebooté par rapport aux nombre total de box de même type déployés en production. Pour ce, nous allons utiliser la notion de scripted field de kibana afin de définir notre nouvel index :

**bootratio** :  $\text{doc}['\text{nb\_stb}'].value / \text{doc}['\text{parc}'].value * 100$  avec comme format «Percentage» (figure ci-dessous)

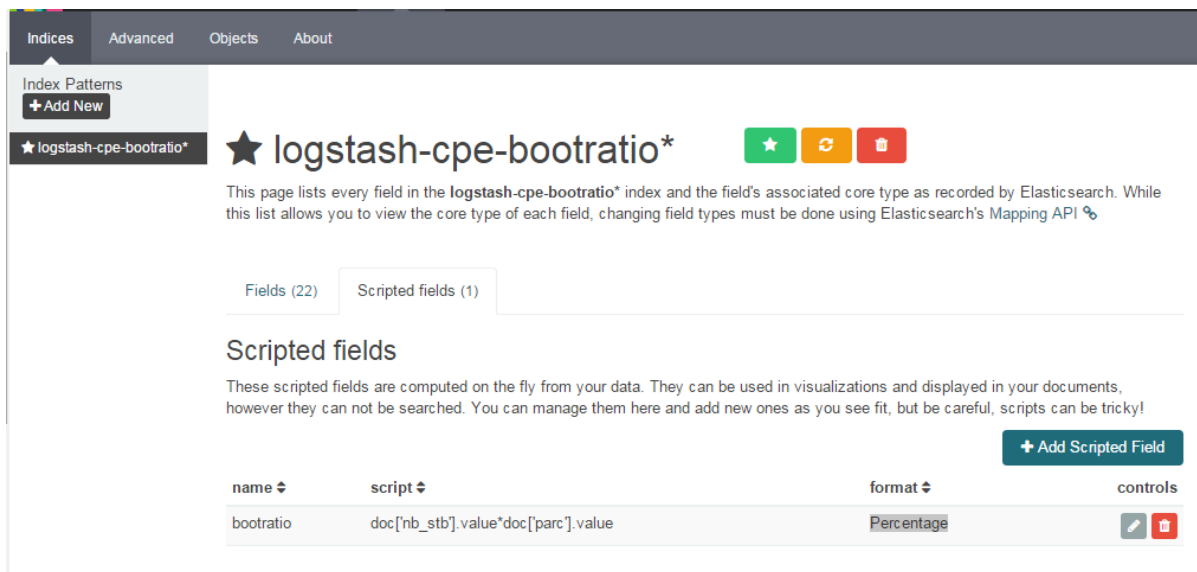


Figure 4 : Création d'un scripted field

A ce stade nous pouvons nous rendre à l'onglet discover de kibana afin de visualiser nos données :

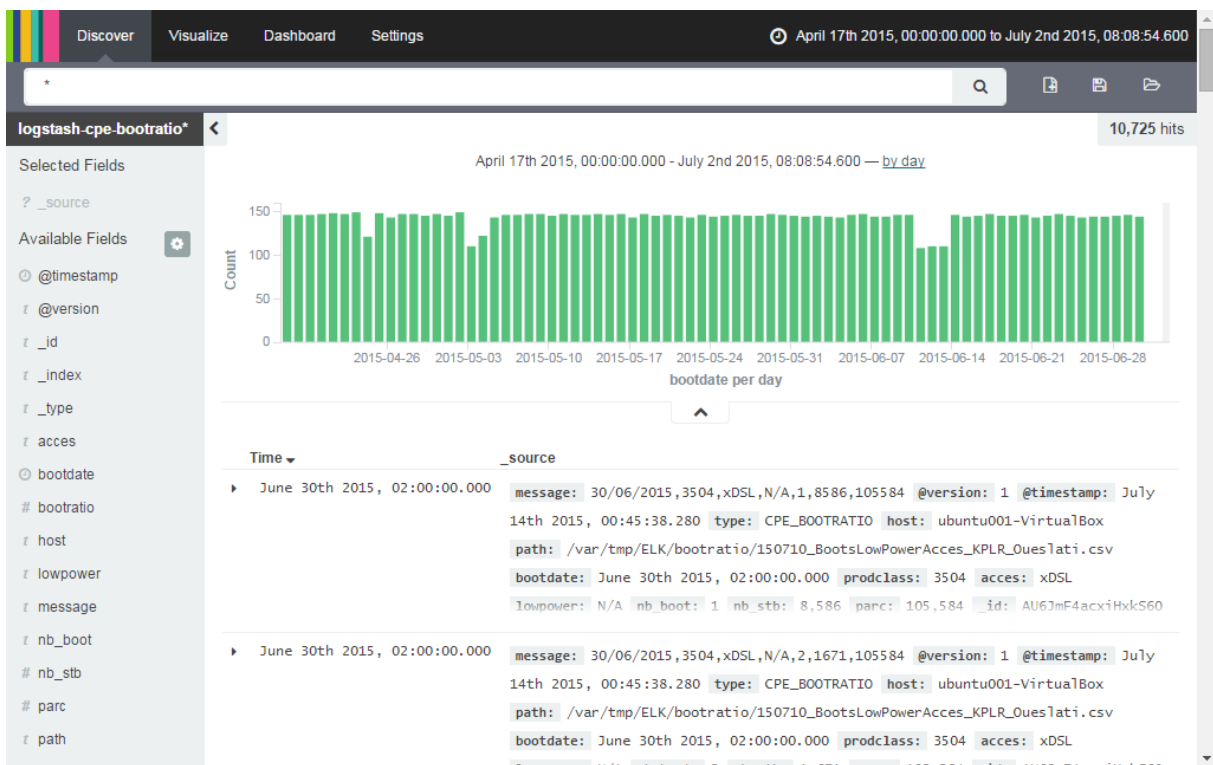


Figure 5 : Onglet Discover de kibana

L'onglet discover nous permet d'effectuer des recherches sur nos données, ainsi on peut n'afficher que certain index.

Passant maintenant à la partie visualisation de kibana : à partir de nos données collectées nous allons tracer un « line chart » qui représente l'évolution journalier du taux de reboot des box de type DBI805BYT.



Dans le graphique ci-dessous nous avons tracé l'évolution du ratio des bbox de type DBI805BYT qui ont rebooté 5 fois et plus par jour, que nous enregistrons sous le nom « nb\_boot:"5 et +" AND prodclass:DBI805BYT » :

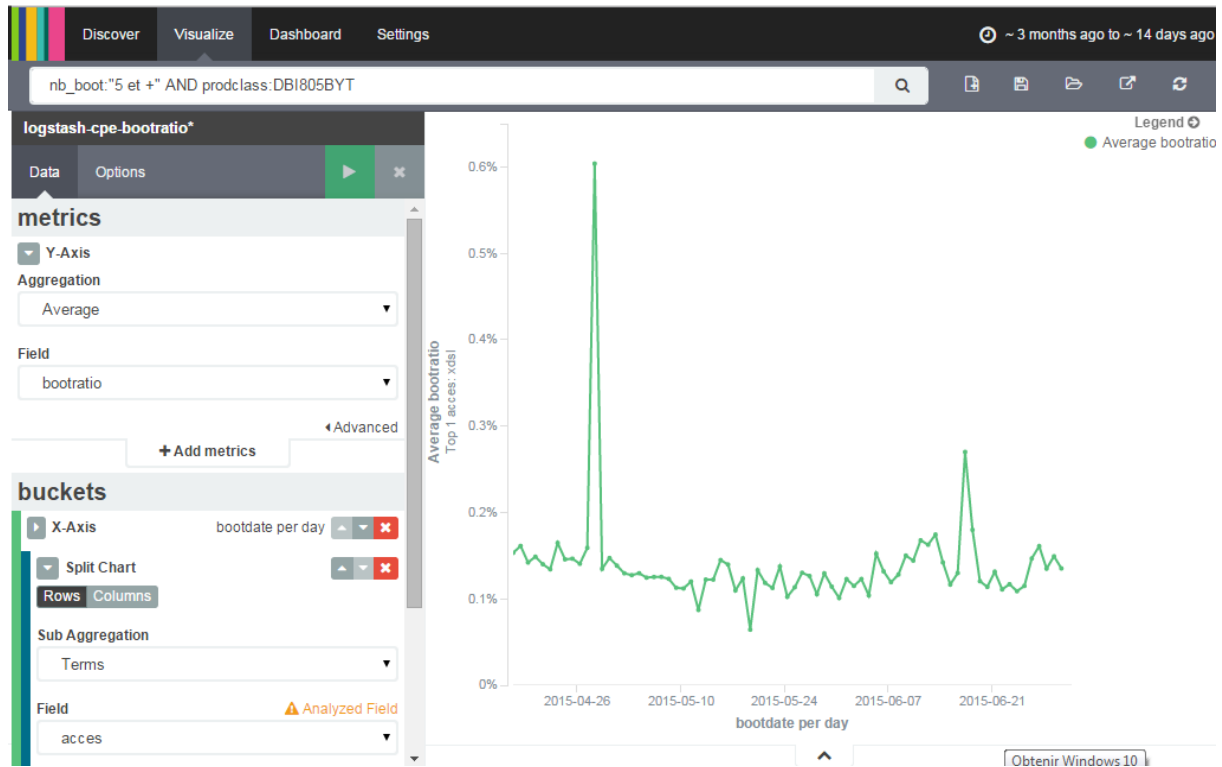


Figure 6 : Graphique Kibana

La recherche Elasticsearch correspondante au graphique de la figure 6 :

```
{
  "query": {
    "filtered": {
      "query": {
        "query_string": {
          "query": "nb_boot:\"5 et +\" AND prodclass:DBI805BYT",
          "analyze_wildcard": true
        }
      },
      "filter": {
        "bool": {
          "must": [
            {
              "range": {
                "bootdate": {
                  "gte": 1429306018617,
```

```

        "lte": 1437082018617
      }
    }
  ],
  "must_not": []
}
}
},
"size": 0,
"aggs": {
  "2": {
    "date_histogram": {
      "field": "bootdate",
      "interval": "1d",
      "pre_zone": "+02:00",
      "pre_zone_adjust_large_interval": true,
      "min_doc_count": 1,
      "extended_bounds": {
        "min": 1429306018617,
        "max": 1437082018617
      }
    },
    "aggs": {
      "4": {
        "terms": {
          "field": "acces",
          "size": 1,
          "order": {
            "1": "desc"
          }
        },
        "aggs": {
          "1": {
            "avg": {
              "script": "doc['nb_stb'].value/doc['parc'].value",
              "lang": "expression"
            }
          }
        }
      }
    }
  }
}
}
}
}
}

```

Maintenant on va mettre en place un tableau de bord qu'on suivra tous les jours et dont les données seront mise à jours en temps réel à chaque insertion de nouveau document.

On commence par créer un tableau de bord qu'on nommera « mon premier tableau de bord ». Dans notre tableau de bord on rajoute les graphique précédemment créés et enregistrés :

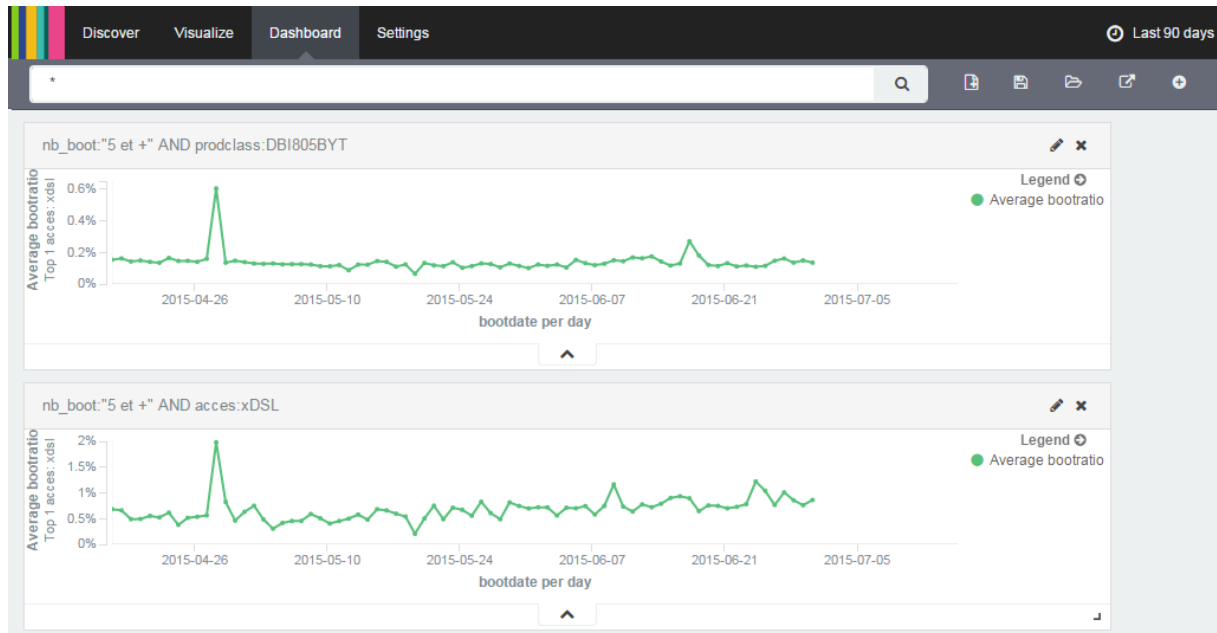


Figure 7 : Tableau de bord

## Conclusion

Dans le cadre de mon activité, la stack ELK me permet d'améliorer le suivi d'exploitation des produits déployés en production en temps réel, avec un gain considérable en capacité à faire sachant que toutes les actions sont automatisées et plus besoin d'analyser les fichiers csv brut.